Tolerant frames

A modal account of fact preservation without ownership

TScert working note

11 November 2026

Abstract

TypeScript programs are not written against an ownership discipline, and TScert imposes none; it must still decide which facts about the heap survive a step. This note gives that reasoning a surface language. Its one new connective is a modality $\Box_\Phi F$, "F holds here and after any change bounded by the effect Φ", which stands where the separating conjunction stands in the frame rule. A fact enters the modality for one of three reasons: the effect is disjoint from what the fact reads; the effect changes what the fact reads only in a direction the fact tolerates; or the invariant annotations on a type fix what the fact reads. Eleven patterns from a TypeScript code base become one-line annotations, and four more show what the language still lacks. Every statement carries one of two marks. [Lean] names the theorem of the TScert development that proves the statement's content, and gives that theorem's scope. [conj.] marks a conjecture. A cited Lean theorem states that content directly about heaps and effects; it does not mention $\Box$.

1 The problem

> **Example 1.** `for (let j = from; j < to; j++) out[j] = xs[j];` The read is in bounds if $to \le |xs|$ holds at each iteration. The array `out` may be an alias of `xs`. Since an element write never shortens an array, the bound survives.

> **Example 2.** `while (i < flags.length) { readRuns(runs); s += flags[i++]; }` The callee calls `runs.openBrackets.splice(...)`, which may shorten that array. The bound $i < |flags|$ survives the call because the array the callee shortens is not `flags`, and because the field `openBrackets` keeps denoting that array.

Example 2 is a frame problem. Example 1 is not: no disjointness is available, and none is required. What the two share is that the step's *effect* is known and the fact is closed under it. The language below makes that the primitive.

2 What exists: variables, locations, stores

Everything below rests on a small inventory, so we state it first.

Variables are not in the heap. Let *Var* be the set of variables, ranged over by x, y, and *Val* the set of values, ranged over by v. An environment $\rho : Var \rightharpoonup Val$ is a finite map from variables to values.

A local binding of the program (a `const`, a `let`, a parameter) is modelled in one of two ways. It is a *cell*, a location of the heap, when a closure can change it or can see it change: a closure writes it, or it is written after the creation of a closure that mentions it. In every other case it is a *variable*, held in ρ. Every `const` is a variable, and so are most `let`s.

For example, in `let i = 0; while (i < k) i++;` the binding `i` is a variable. In `let n = 0; const inc = () => { n++; };` the binding `n` is written by a closure, so it is a cell. In `let m = 0; const get = () => m; m = 1;` the binding `m` is written after `get` was created, so it is a cell too.

A variable is never updated in place. Programs are taken in a normal form where each step reads variables and binds one fresh variable; an assignment `x = e` to a variable binds a fresh variable, which the code after it uses in

place of `x`. For example, the source assignment `i = i + 1` reads the variable i_0 and binds a fresh variable i_1 to $i_0 + 1$; the code after it reads i_1. So a step extends ρ and changes none of its existing bindings.

Two consequences follow. An effect describes changes to the heap, so no effect mentions a variable. A fact that mentions only variables, such as $i < n$, holds after every step once it holds.

Locations. A heap h is a finite map from locations, ranged over by l, to contents. Locations are natural numbers, and the domain of h is the initial segment $\{0, \ldots, |h| - 1\}$: an allocation extends it, and no location is ever removed. The locations of a heap h are of four kinds, by what they hold. Write F, C, E and S for the sets of its locations of each kind; they partition its domain, $\operatorname{dom} h = F \uplus C \uplus E \uplus S$. Each kind belongs to one of two classes, *value* or *store*; effects refer to locations by class.

Field location, in F

One for each own key of each object. It holds the current value of that property. After `const o = { n: 1, m: 2 };` the object has two field locations, one for the key `n` holding 1 and one for the key `m` holding 2; `o.n = 3` changes what the first holds and leaves the second. Class: *value*.

Cell, in C

One for each local binding that is modelled as a cell (the previous paragraph) and for each module binding. It holds the binding's current value. In `let n = 0; const inc = () => { n++; };` the binding `n` has one cell, holding 0; a call `inc()` changes what it holds to 1. Class: *value*.

Element cell, in E

One for each present element of each array. It holds that element's current value. In JavaScript an array literal with nothing between two commas leaves that index empty: `const a = [10, , 30];` makes an array of length 3 with elements at the indices 0 and 2 and none at index 1. An index with no element is a *hole*. This array has two element cells, l_0 holding 10 and l_2 holding 30; the hole at index 1 has no location. `a[0] = 11` changes what l_0 holds. Class: *value*.

Store location, in S

One for each array. It holds the array's *store*: the sequence, in index order, of the array's element cells and holes. For `a` above the store is $\langle l_0, \circ, l_2 \rangle$. `a.push(40)` allocates an element cell l_3 and changes the store to $\langle l_0, \circ, l_2, l_3 \rangle$; `a.length = 1` changes it to $\langle l_0 \rangle$. Class: *store*.

So a heap maps a location to a value or to a store: $h(l) \in \mathit{Val}$ for $l \in F \cup C \cup E$, and $h(l) \in \mathit{Store}$ for $l \in S$. Here $\mathit{Store} = (\mathit{Loc} \uplus \{\circ\})^*$ is the set of finite sequences whose entries are locations or the hole $\circ$, and $\mathit{Loc} = \mathbb{N}$. For a store $\sigma = \langle c_0, \ldots, c_{n-1} \rangle$ we write $|\sigma| = n$ for its length and $\mathit{cells}(\sigma) = \{c_i \mid c_i \neq \circ\}$ for the set of locations it lists. In one line, $h : \mathit{Loc} \rightharpoonup \mathit{Val} \uplus \mathit{Store}$.

A value in *Val* is a scalar, a function, or an object. A *scalar* is `null`, `undefined`, a number, a string, a boolean or a bigint; $\mathit{Scalar} \subseteq \mathit{Val}$ is the set of scalars and $\mathit{Obj} \subseteq \mathit{Val}$ the set of objects. An object is a finite map $v : \mathit{Key} \rightharpoonup F$ from its own keys to field locations, fixed when the object is allocated. The read of a field f of v in a heap h is $h(v(f))$. The object's identity is the map v; what changes is h at the locations $v(f)$. An array is an object with one distinguished field, whose location $s \in S$ is the array's store location. Its *store* is the sequence $h(s) = \langle c_0, \ldots, c_{n-1} \rangle \in \mathit{Store}$, where $c_i \in E$ if index i is present and $c_i = \circ$ if it is a hole. The array's length is n, and its element at a present index i is $h(c_i)$. For `const a = [10, , 30];` the store is $h(s) = \langle l_0, \circ, l_2 \rangle$, the length is 3, $h(l_0) = 10$ and $h(l_2) = 30$.

A whole heap. Take the program `const o = { n: 1, m: 2 }; let k = 0; const inc = () => { k++; }; const a = [10, , 30];` and number the locations in the order they are allocated. After it runs, one heap h it can have is:

location	kind	holds	
0	F	1	the field `n` of `o`
1	F	2	the field `m` of `o`
2	C	0	the cell of `k`
3	S	$\langle 4, \circ, 5 \rangle$	the store of `a`
4	E	10	the element `a[0]`
5	E	30	the element `a[2]`

Here $\operatorname{dom} h = \{0, \ldots, 5\}$, $F = \{0, 1\}$, $C = \{2\}$, $S = \{3\}$ and $E = \{4, 5\}$. The object `o` is the map $\{\mathtt{n} \mapsto 0, \mathtt{m} \mapsto 1\}$ from its keys to their field locations: the value of `o` holds no field contents, and `o.n` is read as $h(0)$, `o.m` as $h(1)$. The array `a` is an object whose distinguished field is the location 3. The environment ρ holds `o`, `inc` and `a`; the

binding `k` is the cell 2. A call `inc()` changes $h(2)$ to 1 and nothing else, and `o.m = 7` changes $h(1)$ to 7 and nothing else.

So an array has two layers that change independently: the store $h(s)$, and what each element cell holds. In the example, `a[0] = 11` changes $h(l_0)$ and leaves $h(s)$ as it is, and `a.push(40)` changes $h(s)$ and leaves $h(l_0)$ and $h(l_2)$ as they are. Each operation writes as follows:

operation	writes	class written
`x = e`, `x` a variable	nothing in the heap	
`x = e`, `x` a cell	the cell	*value*
`o.f = e`	the field location	*value*
`a[i] = e`, index present	the element cell	*value*
`a[i] = e` at a hole or past the end; `push`	the store (longer or filled), and a new cell	*store*
`pop`, `splice`, `a.length = n`	the store (possibly shorter)	*store*

The *value* class is thus "contents": every location that holds something other than a store. The *store* class is "shape": which elements an array has and how many. A fact about a length or about presence reads a store; a fact about what a field or an element contains reads a value location.

Which locations can change. A location never moves or disappears; what changes is its contents. We say that a location l *changes* between two heaps h and h' when $h'(l) \neq h(l)$. Every location can change, through any alias. What bounds the change is the location's *contract*: the set of contents the location may hold, fixed when the location is allocated. A field declared `n: number` has the contract "a number": `o.n = 2` is within it and `o.n = "a"` is outside it. TypeScript's `readonly` restricts one view of a location and makes no location immutable, since another alias may write.

A location is *immutable* exactly when its contract admits a single value. Two kinds of location are immutable. The first is a field declared `@stable`. This is an annotation of TScert, written in a comment on a `readonly` field of a record type: `type Runs = { /** @stable */ readonly openBrackets: number[] }`. It states that the field holds, for the object's whole life, the value the object was created with; the contract of the field's location is that one value. Here the field `openBrackets` of a `Runs` object always holds the same array, whose elements and length may still change. The second is a typed array's length. They belong to the value class like any field, and no admissible step (Section 3) changes them.

3 Model

Worlds and states. A *world* W maps each location to its contract. A contract is a type, and a type denotes a predicate on contents that is step-indexed and world-monotone [1]. A world W' *extends* W when it gives every location of W the contract W gives it; W' may have more locations.

A heap h *meets* a world W when every location of h holds contents within its contract in W; an environment meets W when each of its values does. A *state* is a tuple (ρ, h, W, k): an environment, a heap, a world that ρ and h meet, and a budget $k > 0$ of remaining steps. A state is *well-formed* when every value in ρ meets its type and every element cell that a store lists is a location of h. In the heap of Section 2 the store $\langle 4, \circ, 5\rangle$ lists the locations 4 and 5, and both are in dom h.

A write within a location's contract preserves every typing judgment, whatever aliases exist [1]. This is the one use the note makes of types.

Admissible transitions. An admissible transition is a change of state that respects every contract. Its definition mentions no program. A state $s' = (\rho', h', W', k')$ is *admissible from* a state $s = (\rho, h, W, k)$ when W' extends W and $k' \leq k$. Since s' is a state, h' meets W'; so every location of h holds in h' contents within the contract it had in W. We write $h \rightarrow_\rho h'$ when h and h' are the heaps of two such states and ρ' extends ρ [Lean: `Admissible`].

In the heap of Section 2, the call `inc()` takes h to the heap that differs from h only in holding 1 at location 2. The contract of location 2 is "a number", so this is an admissible transition. A heap holding a string at location 2 is not reached from h by any admissible transition.

Programs enter through a theorem, not through the definition. Let t be a term that is safe at a state s, in the sense of the step-indexed model; a term the type system accepts is. If t runs from s for j steps and reaches a heap h', then h' is the heap of a state that is admissible from s, with budget $k - j$ [Lean: `run`, `run_normal`, `run_throw`, `safe_run`]. So what is proved below of every admissible transition holds of every step, call and callback of typed code.

The relation has four properties, used throughout:

(T1) Preorder
$h \to_\rho h$; and if $h \to_\rho h'$ and $h' \to_\rho h''$ then $h \to_\rho h''$, the worlds composing through the intermediate state's world [Lean: `Admissible.refl`, `Admissible.trans`].

(T2) Growth
If $h \to_\rho h'$ then $|h| \le |h'|$: a transition allocates and never removes a location [Lean: `Admissible.length_le`].

(T3) Fixed objects
Let $v : \mathit{Key} \rightharpoonup F$ be an object value and $h \to_\rho h'$. The domain of v and each location $v(f)$ are the same before and after: the field f of v is read as $h(v(f))$ in h and as $h'(v(f))$ in h'. This holds because v is a value: it does not depend on the heap.

(T4) Immutable locations
If $h \to_\rho h'$, $l < |h|$ and the contract of l admits the single value u, then $h(l) = h'(l) = u$ [Lean: `Admissible.stable_value`].

Changes. A *change* bounds how the contents of one location may differ between two heaps. One of the changes below, grow, must say that the cells a store newly lists are *new* locations. So a change takes a parameter b, a heap length: a location is new when it is at or above b. Formally a change δ is a family of preorders δ_b on contents, one for each $b \in \mathbb{N}$, with $\delta_{b'} \subseteq \delta_b$ for $b \le b'$: a larger b leaves fewer locations new, so it allows less. We write $\delta_b(u, u')$ when δ_b relates the old contents u to the new contents u'.

The parameter is used in two places only. Among the named changes, condition (iv) of grow reads it; keep, init, scalar and any are the same relation for every b. And the action of an effect (below) sets it: a change that starts from a heap h is taken at $b = |h|$, so "new" means allocated by that change. There are five named changes. Recall that $\mathit{Obj} \subseteq \mathit{Val}$ is the set of objects (Section 2). Recall that $\mathit{cells}(\sigma)$ is the set of locations a store σ lists. Write N for the maximal array length. For contents u, u':

$$
\begin{array}{lcl}
\mathsf{keep}_b(u,u') & \iff & u' = u \\
\mathsf{init}_b(u,u') & \iff & u' = u \ \lor \ (u \in \{\mathbf{null}, \mathbf{undefined}\} \land u' \in \mathit{Val}) \\
\mathsf{scalar}_b(u,u') & \iff & u' = u \ \lor \ (u \in \mathit{Val} \setminus \mathit{Obj} \land u' \in \mathit{Val}) \\
\mathsf{grow}_b(u,u') & \iff & \mathsf{scalar}_b(u,u') \ \lor \ (u, u' \in \mathit{Store} \land u \preceq_b u') \\
\mathsf{any}_b(u,u') & \iff & \mathsf{true}
\end{array}
$$

where, for two stores, $\sigma \preceq_b \sigma'$ (read: σ' is a *growth* of σ at b) is the conjunction of four conditions:

(i) $|\sigma| \le |\sigma'|$
(ii) $\forall n.\ \mathit{dense}_n(\sigma) \Rightarrow \mathit{dense}_n(\sigma')$, where $\mathit{dense}_n(\sigma) \iff n \le |\sigma| \land \forall i < n.\ \sigma_i \neq \circ$
(iii) $|\sigma| \le N \Rightarrow |\sigma'| \le N$
(iv) $\mathit{cells}(\sigma') \setminus \mathit{cells}(\sigma) \subseteq \{l \mid l \ge b\}$.

In words: keep allows no change. init lets **`null`** or **`undefined`** be replaced by any value. scalar lets any value that is not an object (a scalar or a function) be replaced by any value. grow also lets a store grow. any allows everything.

A change bounds contents and ignores types. So $\mathsf{scalar}_b(1, \texttt{"a"})$ holds: a number may become a string as far as the change is concerned. What forbids it at a location declared `number` is the contract. A change bounded by an effect is, by its definition below, also an admissible transition, so both bounds apply: the new contents are related to the old ones by the change, and they are within the location's contract. For a field declared `n: number | string`, scalar allows the step from 1 to `"a"` and so does the contract. For a field declared `n: number` the contract forbids it.

Each change allows all that the one before it allows:

$$\mathsf{keep} \sqsubseteq \mathsf{init} \sqsubseteq \mathsf{scalar} \sqsubseteq \mathsf{grow} \sqsubseteq \mathsf{any}.$$

[Lean: `Change.Holds`; `Change.Holds.refl`, `.mono`, `.trans`, `.lower`]

Examples. Let l be the location of `c`, holding **`undefined`**. The assignment in `c ??= build()` changes the contents of l from **`undefined`** to the value v that `build()` returns, and $\mathsf{init}_b(\mathbf{undefined}, v)$ holds. The assignment `o.n = 3` changes the contents of the field location of `n` from 1 to 3, and $\mathsf{scalar}_b(1, 3)$ holds. With $b = 6$, grow_6 relates the store $\langle 4, \circ, 5\rangle$ to $\langle 4, \circ, 5, 6\rangle$ (a push) and to $\langle 4, 6, 5\rangle$ (a write at the hole). It does not relate it to $\langle 4\rangle$, which fails (i), nor to $\langle 4, \circ, 5, 0\rangle$, which fails (iv): 0 is an old location.

Regions. Figure 1 shows the heap of Section 2 and the locations that each region defined in this paragraph denotes in it.

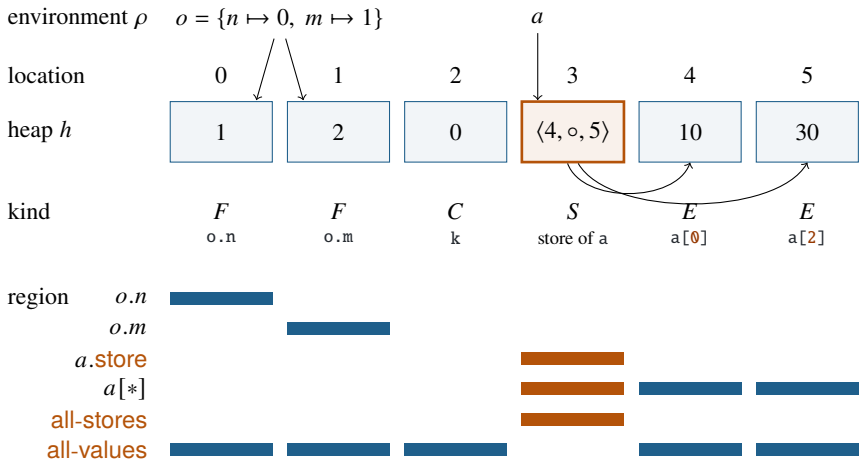


Figure 1: The heap of Section 2 and what each region denotes in it. Top: the object `o` points to its two field locations, and the array `a` to its store location. Middle: the six locations with their contents and kinds; the store at location 3 lists the element cells 4 and 5, with a hole between them. Bottom: each bar marks a location the region denotes; value locations are blue and store locations orange.

A *path* $\pi = x.f_1 \dots f_k$ is a variable followed by field names. Its *read* in (ρ, h) starts from $\rho(x)$ and follows the fields $f_1, \dots, f_k$ in turn through h; the read *fails* when a value on the way is not an object with the next field. A *region* r denotes a set $r(\rho, h)$ of locations of h. There are five forms:

all-stores
: Every location of class store: all-stores$(\rho, h) = S$. In the heap of Section 2 it denotes $\{3\}$.

all-values
: Every location of class value: all-values$(\rho, h) = F \cup C \cup E$. In the heap of Section 2 it denotes $\{0, 1, 2, 4, 5\}$.

π.store
: The store location of the one array that π reads: $(\pi.\text{store})(\rho, h) = \{s\}$, where s is that array's store location. Example: a.store denotes $\{3\}$.

$\pi[*]$
: That store location together with the element cells its store lists: $(\pi[*])(\rho, h) = \{s\} \cup \mathit{cells}(h(s))$. Example: $a[*]$ denotes $\{3, 4, 5\}$.

$\pi.f$
: The field location for the key f of the one object v that π reads: $(\pi.f)(\rho, h) = \{v(f)\}$. Examples: $o.n$ denotes $\{0\}$ and $o.m$ denotes $\{1\}$.

The first two regions are named for what they cover: all the stores of the heap, and all its value locations. In TScert's `Modifies` annotation they are written `stores` and `values`. A region at a path whose read fails denotes the empty set: if `z` is **`null`**, then $z.\mathit{open}$.store denotes $\emptyset$.

Effects. An *effect* says where a step may change the heap and by how much. It is a finite set of entries $r : \delta$, each a region and a change:

$$\begin{aligned} \Phi &::= \{\, r_1 : \delta_1,\ \dots,\ r_k : \delta_k \,\} \\ r &::= \text{all-stores} \mid \text{all-values} \mid \pi.\text{store} \mid \pi[*] \mid \pi.f \\ \delta &::= \text{keep} \mid \text{init} \mid \text{scalar} \mid \text{grow} \mid \text{any} \end{aligned}$$

Two examples. The effect {all-values : any, all-stores : grow} reads: every location that is not a store may change arbitrarily, and every array may grow and none may shrink. It is the effect of an element write `out[j] = v` whose target array is unknown. The effect {all-values : any, y.store : any} reads: every location that is not a store may change arbitrarily, the one array y may change arbitrarily, and every other array is unchanged. It is the effect of a call that may shorten y, as `y.splice(...)` does.

The action of an effect. An effect Φ denotes a relation $\xrightarrow{\Phi}_\rho$ between heaps, its *action*. We say that h' is reached from h by a change *bounded by* Φ when $h \xrightarrow{\Phi}_\rho h'$:

$$h \xrightarrow{\Phi}_\rho h' \iff h \to_\rho h' \ \wedge\ \forall l < |h|.\ \big(h'(l) = h(l) \ \vee\ \exists (r : \delta) \in \Phi.\ l \in r(\rho, h) \wedge \delta_{|h|}(h(l), h'(l))\big).$$

In words: the transition is admissible; and every location of h either holds what it held, or lies in the region of some entry and changed within that entry's change. Three points of the definition matter later. The regions are read in the heap h the change starts from, as a dynamic frame is [9]. A location that no entry covers is unchanged. Locations at or above $|h|$ are allocated by the transition and are unconstrained.

For example, let h be the heap of Section 2 and let h' be h after `a.push(40)`: $h'(3) = \langle 4, \circ, 5, 6\rangle$, $h'(6) = 40$, and h' agrees with h on 0, 1, 2, 4 and 5. Then $h \xrightarrow{\{\text{all-stores:grow}\}}_\rho h'$: location 3 is in all-stores(ρ, h) and changed within grow$_6$, the locations 0, 1, 2, 4 and 5 are unchanged, and 6 is new. After `a.length = 1` the store is $\langle 4\rangle$, a change that is not within grow; that transition is bounded by $\{a.\text{store} : \text{any}\}$ and is not bounded by $\{\text{all-stores} : \text{grow}\}$.

Order, top, rigidity. We write $\Phi \sqsubseteq \Psi$ when the action of Φ is included in the action of Ψ, for every ρ. We write $\top$ for $\{\text{all-values} : \text{any},\ \text{all-stores} : \text{any}\}$, the effect of a step about which nothing is known; every effect is below $\top$.

An effect Φ is *rigid* when its action is transitive: if $h \xrightarrow{\Phi}_\rho h'$ and $h' \xrightarrow{\Phi}_\rho h''$ then $h \xrightarrow{\Phi}_\rho h''$. Rigidity is what lets a fact that survives one change bounded by Φ survive any number of them.

Rigid: class regions
An effect whose regions are all-stores and all-values is rigid at every named change. A change below any keeps each location in its class, and any absorbs any change that follows it.

Rigid: a store at a rigid path
$\{\pi.\text{store} : \delta\}$ is rigid on the states where π is rigid (Section 4): the path then reads the same store in h and in h'.

Not rigid: cells of a store changed at any
$\{x[*] : \text{any}\}$ is not rigid. A first change may make the store of x list a location that existed before, such as location 0 in the heap of Section 2. A second change, whose region is read in the new heap, may then change location 0, which the region read in h does not contain.

The effect of a command. The judgment $c : \Phi \mid \Phi^e$ states two bounds on a command c run from (ρ, h). If c returns, in a heap h', then $h \xrightarrow{\Phi}_\rho h'$. If c throws, in a heap h', then $h \xrightarrow{\Phi^e}_\rho h'$. We write $c : \Phi$ when Φ^e is $\top$. For example `out[j] = v` : $\{\text{all-values} : \text{any},\ \text{all-stores} : \text{grow}\}$. This judgment is what a `Modifies` annotation states and what an effect system derives [Lean: `Post`; `checkRequests_footprint_at`, `checkRequests_regional_at` for graph code].

4 The assertion language

Syntax. Assertions F, G are built from atoms a:

F, G	::=	$a \mid F \wedge G \mid g \vee F \mid \exists v.\ F$	g an invariant assertion; v a logical variable
	$\mid$	$\pi \,\#\, \pi' \mid \text{apart}(x, y) \mid \text{fresh}(x)$	separation and freshness
	$\mid$	$\Box_\Phi F$	the modality

Atoms. An atom is a fact of a first-order language over the heap. An atom is true only where every path it reads is defined. There are five forms:

Linear inequality
An inequality between linear terms over variables and lengths $|\pi|$ of arrays at paths. Examples: $i < |\mathit{flags}|$ and $\mathit{to} \le |\mathit{xs}|$.

Equality $\pi \doteq v$
The path π reads the value v. Example: $z \doteq$ `null`.

Density $\mathsf{dense}(x, n)$

The first n indices of the array x are present. For `a = [10, , 30]`, $\mathsf{dense}(a, 1)$ holds and $\mathsf{dense}(a, 2)$ does not.

Kind of contents

The contents of a cell are of a stated kind, such as a number or an object. Example: the contents of $x.f$ are a number.

Bounded quantification $\mathsf{every}(z, t, \varphi)$

Each element of the array z at an index below t satisfies φ. For `a = [10, , 30]`, $\mathsf{every}(a, 1, v > 0)$ holds.

Satisfaction. An assertion is interpreted at a triple (ρ, m, h): an environment, an *entry mark*, and a heap. The entry mark $m \le |h|$ is the length the heap had when the current function invocation started; the locations at or above m are the ones the invocation has allocated. We write $\rho, m, h \models F$. Atoms read ρ and h only, and the connectives $\wedge$, $\vee$, $\exists$ have their usual meaning. The other four forms are:

$$
\begin{array}{lcl}
\rho, m, h \models \pi \mathbin{\#} \pi' & \iff & \pi \text{ and } \pi' \text{ read arrays with different store locations,} \\
\rho, m, h \models \mathsf{apart}(x, y) & \iff & x \mathbin{\#} y, \text{ and the stores of } x \text{ and } y \text{ list no common element cell,} \\
\rho, m, h \models \mathsf{fresh}(x) & \iff & \text{the store location of } \rho(x) \text{ is at or above } m, \\
\rho, m, h \models \Box_\Phi F & \iff & \text{for all } h', \text{ if } h \xrightarrow{\Phi}_\rho h' \text{ then } \rho, m, h' \models F.
\end{array}
$$

So $\Box_\Phi F$ says: F holds in every heap reached from here by a change bounded by Φ. Since $h \xrightarrow{\Phi}_\rho h$, it says in particular that F holds here.

Two examples, in the heap h of Section 2, where $|a| = 3$. First, $\rho, m, h \models \Box_{\text{all-values:any, all-stores:grow}}(3 \le |a|)$: a change bounded by this effect leaves the store of a or grows it, so its length stays at least 3. Second, $\rho, m, h \not\models \Box_{a.\text{store:any}}(3 \le |a|)$: the heap after `a.length = 1` is reached by a change bounded by this effect, and there $|a| = 1$. In a function `f() { const xs = [1]; ...}`, $\mathsf{fresh}(xs)$ holds after the declaration: the store of `xs` was allocated by this invocation.

Assertions describe the whole heap. They can be duplicated, and holding one confers no permission to write.

Invariant assertions and rigid paths. Two abbreviations:

$$
F \text{ is } \textit{invariant} \triangleq F \vdash \Box_\top F \qquad \mathsf{rigid}(\pi) \triangleq \exists v.\ \Box_\top(\pi \doteq v).
$$

An invariant assertion, once true, is true after every admissible transition. A fact about variables alone, such as $i < n$, is invariant. So is $x \mathbin{\#} y$ for two variables: an array's store location is fixed when the array is allocated. A path is rigid when it reads the same value after every admissible transition. A path through fields whose contracts admit one value is rigid: `runs.openBrackets` is rigid when `openBrackets` is declared `@stable`. In $\mathsf{rigid}(\pi)$ the witness v is a logical variable, so it is the same value before and after the change.

> *Remark* 3 (Reading). $\Box_\Phi F$ is the weakest liberal precondition of F for the command that makes one arbitrary admissible change bounded by Φ: $\Box_\Phi F = \mathit{wlp}(\mathsf{havoc}\,\Phi, F)$, the reading a verifier gives a call when it replaces it by its `modifies` clause. For a rigid Φ it is the weakest assertion that is stronger than F and stable under $\xrightarrow{\Phi}$: the floor of explicit stabilisation [11], at a rely denoted by an effect.

> **Proposition 4** (Laws). *[conj.]*
>
> *(a)* $\Box_\Phi F \vdash F$.
>
> *(b)* $\Box_\Phi(F \wedge G) \dashv\vdash \Box_\Phi F \wedge \Box_\Phi G$; *if* $F \vdash G$ *then* $\Box_\Phi F \vdash \Box_\Phi G$.
>
> *(c)* *If* $\Phi \sqsubseteq \Psi$ *then* $\Box_\Psi F \vdash \Box_\Phi F$.
>
> *(d)* $g \vee \Box_\Phi F \vdash \Box_\Phi(g \vee F)$ *for invariant* g.
>
> *(e)* $\Box_\Phi F \vdash \Box_\Phi \Box_\Phi F$ *for rigid* Φ.

In words: (a) what holds after every change holds now; (b) the modality distributes over conjunction and is monotone; (c) a fact that survives a larger effect survives a smaller one; (d) an invariant disjunct can be moved under the modality; (e) under a rigid effect, a fact that survives one change survives a change after a change. Each is immediate:

(a) from reflexivity of the action, (b), (c) and (d) from the definitions, (e) from transitivity. For effects over class regions transitivity is `Closed.Holds.trans_on`; for a store at a path it is `ArgumentStoreRegionsWithin.trans`, under that theorem's premise that the path resolves to the same store in both heaps.

5 Entering the modality

A fact enters the modality by naming what it reads and how much change of it the fact tolerates.

Definition 5 (Support)**.** A *support* for an assertion F is a finite set S of pairs $r : \delta$, a region and a change, with this property: for all ρ, m, h, h', if $\rho, m, h \models F$, $h \rightarrow_\rho h'$, and every location $l < |h|$ in a region $r(\rho, h)$ of a pair $(r : \delta) \in S$ satisfies $\delta_{|h|}(h(l), h'(l))$, then $\rho, m, h' \models F$. Locations outside the regions of S may change in any admissible way.

For example $\{x.\mathsf{store} : \mathsf{grow}\}$ supports $n \le |x|$: the fact reads the store of x and nothing else in the heap, and it tolerates any growth of that store.

Definition 6 (Respect)**.** Let K be an assertion. An effect Φ *respects* a support S under K, written $\Phi \triangleleft_K S$, when for every pair $(r : \delta) \in S$ and every entry $(r' : \gamma) \in \Phi$, either $\gamma \sqsubseteq \delta$, or K entails that r and r' denote disjoint sets.

For example $\{\mathsf{all\text{-}values} : \mathsf{any},\ \mathsf{all\text{-}stores} : \mathsf{grow}\}$ respects $\{x.\mathsf{store} : \mathsf{grow}\}$ under true. The entry $\mathsf{all\text{-}stores} : \mathsf{grow}$ meets the first condition, $\mathsf{grow} \sqsubseteq \mathsf{grow}$. The entry $\mathsf{all\text{-}values} : \mathsf{any}$ meets the second: a store location is no location of class value.

Theorem 7 (Tolerant frame)**.** *If S supports F and $\Phi \triangleleft_K S$, then $K \wedge F \vdash \Box_\Phi F$. [Lean: `stable_sound`, with `frame_sound` for the supports of cell, link, density and element-kind facts]*

Proof. Let $h \xrightarrow{\Phi}_\rho h'$ and $l \in r(\rho, h)$ with $(r : \delta) \in S$. Either $h'(l) = h(l)$, and δ is reflexive, or some $(r' : \gamma) \in \Phi$ covers l and bounds its change by γ. Then r and r' are not disjoint, so $\gamma \sqsubseteq \delta$. □

The theorem has two known rules as its extreme cases. When every change in the support is keep, the condition is that the regions a command writes are disjoint from the regions a formula reads: the frame rule of region logic [4]. When K separates no pair, the condition is that the assertion is stable under an interfering action [8]. The theorem chooses between the two at each pair.

A fact may have several supports and none of them greatest. On well-formed arrays, $\mathsf{apart}(x, y)$ is supported by $\{x[*] : \mathsf{grow},\ y[*] : \mathsf{keep}\}$: a store that grows lists only the cells it listed and new ones. It is also supported by the mirror image, $\{x[*] : \mathsf{keep},\ y[*] : \mathsf{grow}\}$. It is not supported by $\{x[*] : \mathsf{grow},\ y[*] : \mathsf{grow}\}$: both stores may come to list the same new cell.

Proposition 8 (Axioms of the atoms)**.** *For variables x, y, z, w, a rigid effect in (Rigid), and a an atom that reads, besides variables, only what is displayed:*

(Grow)	$n \le \lvert x\rvert \vdash \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ \mathsf{all\text{-}stores}:\mathsf{grow}}\ (n \le \lvert x\rvert)$	
(Dense)	$\mathsf{dense}(x, n) \vdash \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ \mathsf{all\text{-}stores}:\mathsf{grow}}\ \mathsf{dense}(x, n)$	
(Sep)	$x \# y \wedge a(\lvert x\rvert) \vdash \Box_{\mathsf{all\text{-}values}:\mathsf{any},\ y.\mathsf{store}:\mathsf{any}}\ a(\lvert x\rvert)$	
(Every)	$\mathsf{apart}(x, z) \wedge \mathsf{every}(z, t, \varphi) \vdash \Box_{x[*]:\mathsf{any}}\ \mathsf{every}(z, t, \varphi)$	*t a variable, φ over scalars*
(Init)	$a(\lvert\pi\rvert) \vdash \Box_{\mathsf{all\text{-}values}:\mathsf{init}}\ a(\lvert\pi\rvert)$	*π through fields holding objects*
(Rigid)	$\mathsf{rigid}(\pi) \wedge a(\lvert\pi\rvert) \vdash \Box_\Phi\ a(\lvert\pi\rvert)$	*if $a(\lvert y\rvert) \vdash \Box_\Phi\ a(\lvert y\rvert)$ for a fresh y*
(View)	$a(\mathsf{view}(w)) \vdash \Box_\top\ a(\mathsf{view}(w))$	*$\mathsf{view}(w)$ a typed array's length*

Each axiom, in words, with the Lean theorem that proves its content. The Lean development proves each for the steps TScert has, not for an arbitrary transition bounded by the displayed effect.

(Grow)
A lower bound on a length survives any change that only grows arrays. It mentions no separation: it is Example 1. Lean: `termValue_grown`, for defined terms on preserved paths.

(Dense)
A dense prefix survives any change that only grows arrays. Lean: `dense_support`.

(Sep)
A fact about the length of x survives an arbitrary change of another array y. It is Example 2. Lean: `regionFirst_kept`, at a call.

(Every)
A fact about each element of z survives an arbitrary change of an array x that shares no element cell with z. Lean: `writeAtoms_holds`, for an element write at a checked index.

(Init)
A fact about a length survives a change that only initialises: init changes no location that holds an object, a number or a store. It is Theorem 7 at a support that tolerates init on the fields of the path. Lean: `ofClosed_holds`.

(Rigid)
A fact that holds of an array named by a variable holds of the array at a rigid path. The substitution of the witness for y is capture-avoiding. Lean: `stableFirst_frame`, for `@stable` fields on normal return.

(View)
A fact about a typed array's length survives every change: that length is an immutable location, by (T4). Lean: the rule `view-stable`.

6 Programs

Triples. A triple $\{P\}\ c\ \{Q \mid Q^e\}$ states partial correctness of a command c. If c runs from a state satisfying P and returns, the state it returns in satisfies Q. If it throws, the state at its handler satisfies Q^e. We write $\{P\}\ c\ \{Q\}$ when Q^e is true. For example $\{\mathsf{true}\}\ x = [\,]\ \{|x| = 0\}$.

The posts Q and Q^e are *two-state*: in them $\mathsf{old}(e)$ denotes the value of e in the environment and heap that c started in. For example `x.push(v)` has the post $|x| = \mathsf{old}(|x|) + 1$.

The variable that a step binds is fresh (Section 2). So an assertion R over the variables in scope before c is unaffected by the binding.

Rules.

$$\textsc{Frame}\ \frac{\{P\}\ c\ \{Q \mid Q^e\} \qquad c : \Phi \mid \Phi^e}{\{P \wedge \Box_\Phi R \wedge \Box_{\Phi^e} R^e\}\ c\ \{Q \wedge R \mid Q^e \wedge R^e\}} \qquad \textsc{Frame}^+\ \frac{\{P\}\ c\ \{Q\} \qquad c : \Phi \qquad \Phi\ \text{rigid}}{\{P \wedge \Box_\Phi R\}\ c\ \{Q \wedge \Box_\Phi R\}}$$

$$\textsc{Loop}\ \frac{\{I\}\ c\ \{I\} \qquad c : \Phi \qquad \Phi\ \text{rigid}}{\{I \wedge \Box_\Phi R\}\ \mathtt{loop}\ c\ \{I \wedge \Box_\Phi R\}} \qquad \textsc{Call}\ \frac{f(\vec{p}) : \{P\}\ f\ \{Q\} : \Phi}{\{P[\vec{x}/\vec{p}] \wedge \Box_{\Phi[\vec{x}/\vec{p}]} R\}\ y = f(\vec{x})\ \{Q[\vec{x}/\vec{p}] \wedge R\}}$$

$$\textsc{Alloc}\ \frac{x\ \text{fresh} \qquad y\ \text{ranges over the arrays in scope}}{\{\mathsf{true}\}\ x = [\,]\ \{\mathsf{fresh}(x) \wedge |x| = 0 \wedge \textstyle\bigwedge_y \mathsf{apart}(x, y)\}}$$

$$\textsc{Seq-Eff}\ \frac{c_1 : \Phi \qquad c_2 : \Psi \qquad \Psi \rightsquigarrow_{ev} \Psi' \qquad \Phi \cup \Psi' \sqsubseteq \Phi}{c_1; c_2 : \Phi}$$

$$\textsc{Fresh-Eff}\ \frac{c : \Phi \cup \{x.\mathsf{store} : \delta\} \qquad \mathsf{fresh}(x)\ \text{holds before}\ c}{c : \Phi\ \ \text{from the entry}}$$

The rules one by one:

Frame
The frame rule, with $\wedge\, \Box_\Phi$ where separation logic has $*$: a fact R that survives every change bounded by the effect of c holds after c. Its soundness is the definition of the modality and of $c : \Phi \mid \Phi^e$. It holds for a compound c whose effect is stated at its entry, with the variables c binds renamed apart from those of R and Φ.

Frame$^+$
For a rigid effect the frame keeps its modality after c, by law (e).

Loop
As Frame$^+$, for any number of iterations. Here c is one iteration together with the evaluation of its guard; every exit leaves from a state satisfying I; the values the loop carries from one iteration to the next are variables of I; and R mentions none of them.

Call
The frame rule at a call. The actual arguments $\vec{x}$ are variables, hence fixed values. The substitution $[\vec{x}/\vec{p}]$ is capture-avoiding, and Q may mention the result y.

Alloc
A new array is fresh, empty, and apart from every well-formed array y that exists before the step. These separations are facts of the state after the step. Later steps keep or lose them like any other fact.

Seq-Eff
The effect of a sequence, stated at the entry of the sequence. The effect Ψ of the second command is read where that command starts, so it is first *rebased* to the entry (below).

Fresh-Eff
A change to an array that the command itself allocated is no part of the command's effect, seen from its entry.

Definition 9 (Rebasing)**.** Let Ψ be an effect read at an intermediate heap and Ψ' an effect read at the entry heap. $\Psi \rightsquigarrow_{ev} \Psi'$ holds when every location of the entry heap that Ψ covers at a change δ is covered by Ψ' at δ or at a wider change. The evidence ev records, for each value that Ψ names, the entry path it was read from, and that this path reads the same in both heaps.

Three cases follow from the definition. A region at a rigid path rebases to itself. The store of a fresh array is no location of the entry heap, so its entry rebases to nothing: this is Fresh-Eff. Any other region, and the cells of a fresh store, widen to their class. For example, in `f(o) { const xs = o.items; xs.push(1); }` the push has the effect $\{xs.\mathsf{store} : \mathsf{grow}\}$ where it starts. If $o.items$ is rigid, the effect of `f` at its entry is $\{o.items.\mathsf{store} : \mathsf{grow}\}$. Otherwise it widens to $\{\mathsf{all\text{-}stores} : \mathsf{grow}\}$ [Lean: `Post.compose_at, rebaseAll, rebase_sound, checkFresh_sound`].

Example 10 (Freshness bounds effects, not facts)**.** Let `shrink(xs)` pop its argument, `shrink` : $\{\mathsf{all\text{-}values} : \mathsf{any}, xs.\mathsf{store} : \mathsf{any}\}$, and `f() { const xs = [1]; shrink(xs); return xs[0]; }`. By Fresh-Eff, `f` : $\{\mathsf{all\text{-}values} : \mathsf{any}\}$: a caller of `f` sees no change to any array. Inside `f` the fact $0 < |xs|$ is not derivable under $\Box_{xs.\mathsf{store}:\mathsf{any}}$, so it does not hold after the call `shrink(xs)`, and the read `xs[0]` is refused. Freshness gives separations, by Alloc. It does not put any fact under a modality.

Proposition 11 (Reliance)**.** *Let a native operation alternate its own steps with calls of callbacks, every one of effect bounded by a rigid Φ, and observe the heap only between them. If $\Box_\Phi F$ holds when it starts, F holds at each observation. [Lean: `rely_chain`; `map_rely`, `elems_rely` for `map`]*

For example take `xs.map(f)`, where `map` reads $n = |xs|$ once at its start. Let $F = \mathsf{dense}(xs, n)$ and $\Phi = \{\mathsf{all\text{-}values} : \mathsf{any}, \mathsf{all\text{-}stores} : \mathsf{grow}\}$. If `f`, with all it calls, has an effect bounded by Φ, then by (Dense) and the proposition F holds each time `map` reads an element. So the result of `xs.map(f)` is dense when `xs` is.

Exact posts. A step may state an exact two-state relation, such as $|x| = \mathsf{old}(|x|) + 1$ for `x.push(v)`. This is a strong update on shared state, and it needs no ownership. The post constrains the written location only. Every other fact crosses the step by Frame, and that rule accounts for aliasing. For example, let y be an array that may be x. After `x.push(v)` the lower bound $n \le |y|$ still holds, by (Grow). The equality $|y| = n$ does not: it is false when y is x.

7 Transfer along edges

A node of a control-flow graph has one *exit edge* for each way it completes: a normal return, each branch it can take, and a throw to a handler. An exit edge e carries four things:

The effect Φ_e
The effect of the node when it completes through e.

The post Q_e
A sound two-state post of the node on that exit. It includes the binding of the result, or of the thrown value.

The condition B_e
The condition under which a branch is taken; true on other edges.

The renaming θ_e
A map from the variables of the node to the variables of the target of e.

In Example 14 below, the true exit of the test `j < to` has $\Phi_e = \emptyset$, $B_e = (j < to)$ and the identity renaming; the back edge has $\theta_e = [j \mapsto j']$.

The *knowledge* Σ at a point of the graph is a finite set of assertions. A state s satisfies it at an entry mark m, written $s \models_m \Sigma$, when it satisfies each of them.

Definition 12 (Checked transfer)**.** Check(c, Σ, e, A) holds when there is a finite set $K_e \subseteq \Sigma$ with a derivation of $\Sigma \vdash \Box_{\Phi_e} F$ for each $F \in K_e$, and a derivation of

$$\mathsf{old}(\Sigma) \ \wedge \ Q_e \ \wedge \ B_e \ \wedge \ K_e \ \vdash \ \theta_e^* A,$$

where old(Σ) is Σ at the node's entry state, $\theta_e^* A$ is A with the target's variables replaced by the node's, and the entailment includes that every path A reads is defined.

Theorem 13 (Transfer)**.** *[conj. as one statement; proved for TScert's checker edge by edge, `ControlFlow` with `HeapKnowledge`'s transport] If s is well-formed, $s \models_m \Sigma$, the node c runs from s through exit e to s', and* Check(c, Σ, e, A)*, then $\theta_e(s') \models_m A$.*

The proof is Frame for K_e and consequence for the rest. The entry mark m is fixed for the invocation, and old refers to the state at the entry of the node.

Sound graphs. Annotate each point of a graph with knowledge. The annotated graph is *sound* when its entry state satisfies the knowledge at the entry, and Check holds on every edge against the knowledge at the edge's target. Merges and loop heads are targets like any other.

The forward pass. A forward pass is a way of *computing* annotations that satisfy these checks. At a node it keeps the facts it can derive under the modality, and adds what the post gives. At a merge it keeps what every incoming edge establishes. At a loop head one pass cannot compute the knowledge: the knowledge is supplied, and checked on the back edges like on any other edge.

The derivations are those of Propositions 4 and 8 together with linear arithmetic. The system is sound. It is deliberately incomplete for semantic entailment.

Example 14 (Example 1, in full)**.** With the loop variable a parameter of the head, and $\Sigma_H(j) = \{0 \le j,\ to \le |xs|\}$:

	node or edge	effect	obligation and its reason
1	entry $\to H(from)$		$0 \le from,\ to \le \lvert xs\rvert$ given
2	$H(j)$: `j < to` true	$\emptyset$	$B = (j < to)$; all of Σ_H kept, $\emptyset$ changes nothing
3	v = `xs[j]`	$\emptyset$	the read: $0 \le j < to \le \lvert xs\rvert$
4	`out[j] = v`	all-values:any, all-stores:grow	$to \le \lvert xs\rvert$ by (Grow); $0 \le j,\ j < to$ invariant
5	$j' = j + 1$	$\emptyset$	$Q = (j' = j + 1)$
6	back edge, $\theta = [j \mapsto j']$		$\theta^*\Sigma_H = \{0 \le j',\ to \le \lvert xs\rvert\}$ from $0 \le j$, Q, row 4
7	$H(j)$: `j < to` false	$\emptyset$	exit with $\Sigma_H \wedge \neg(j < to)$

No row mentions whether `out` is an alias of `xs`.
Now suppose the invariant also had an upper bound, $|xs| \le n$. Row 4 loses it. No axiom puts an upper bound under $\Box_{\text{all-stores:grow}}$, and the bound is false after the write when `out` is `xs` and $j = |xs|$.
The upper bound is kept under two additions. The knowledge Σ_H includes apart(out, xs). And the effect

of the write is stated at its region, $\{out[*] : \mathsf{grow}\}$. Then the bound is kept by (Sep). The separation itself becomes an obligation of rows 1 and 6.

8 The patterns

Lower bound under aliasing (Ex. 1)	$to \le \lvert xs\rvert \vdash \Box_{\mathsf{all\text{-}values:any},\,\mathsf{all\text{-}stores:grow}}(to \le \lvert xs\rvert)$	(Grow)
Bound across a rewriting call (Ex. 2)	$flags\ \#\ y \wedge i < \lvert flags\rvert \vdash \Box_{\mathsf{all\text{-}values:any},\,y.\mathsf{store:any}}(i < \lvert flags\rvert)$, y standing for the rigid *runs.openBrackets*	(Sep), (Rigid)
Storage the function allocated	the write is no part of the function's effect; $\mathsf{apart}(x, y)$ for older y	Fresh-Eff, Alloc
Write-once cache, `c ??= build()`	$a \vdash \Box_{\Phi_{build} \cup \{\mathsf{all\text{-}values:init}\}}\, a$, by Theorem 7 at a support respected by Φ_{build} and tolerating init	(Init)
Reliance during callbacks	$\mathsf{dense}(xs, n) \vdash \Box_{\mathsf{all\text{-}values:any},\,\mathsf{all\text{-}stores:grow}}\mathsf{dense}(xs, n)$	Prop. 11
Quantified fact across a write	$\mathsf{apart}(a, z) \wedge \mathsf{every}(z, t, \varphi) \vdash \Box_{a[*]:\mathsf{any}}\mathsf{every}(z, t, \varphi)$	(Every)
Structures built in step	$\{\lvert a\rvert = \lvert b\rvert \wedge a\ \#\ b\}$ `a.push(v)` $\{\lvert a\rvert = \mathsf{old}(\lvert a\rvert) + 1 \wedge \lvert b\rvert = \mathsf{old}(\lvert b\rvert)\}$ post, (Sep)	
Separation where a value exists	$(z \doteq \mathtt{null} \vee xs\ \#\ z.open) \wedge a(\lvert xs\rvert) \vdash \Box_{\mathsf{all\text{-}values:any},\,z.open.\mathsf{store:any}}\, a(\lvert xs\rvert)$; at `null` the region is empty	
Fact through a field	$\mathsf{rigid}(x.f) \wedge n \le \lvert x.f\rvert \vdash \Box_{\mathsf{all\text{-}values:any},\,\mathsf{all\text{-}stores:grow}}(n \le \lvert x.f\rvert)$; fails without rigid	(Rigid)
Typed-array length	$i < \mathsf{view}(w) \vdash \Box_{\top}(i < \mathsf{view}(w))$	(View)
Fact at a handler	$\Box_{\Phi^e} R^e$	Frame

Four patterns are outside the language. Three need a form it lacks:

A fact about a field of each element

Example: `rows[i].n` ≥ 0 for every i. It needs a region $z[*].n$: a path from each element cell the store lists.

Overlapping typed-array views

Two views are distinct objects over one buffer, so # on their stores says nothing about their elements. They need regions of buffer cells with offsets.

A callback that writes what it captured

Example: `walk(xs, v => out.push(v))`. Each use has a closed effect. A reusable specification of `walk` needs an effect variable, $\forall\varepsilon.\ \Phi \cup \varepsilon$.

The fourth needs no new form. It needs a separation that the rules do not establish:

A record replaced on each loop pass

The new record must be separated from the loop's iterator on every back edge. Alloc states separation only against what is in scope at the allocation.

9 Relation to separation logic

The language takes from separation logic [10] the frame rule and, in Alloc and Fresh-Eff, the principle that what a computation allocates is outside every frame of its callers. It has no separating conjunction and no resource model: assertions describe the whole heap and are duplicable. In Frame the effect is stated, where separation logic derives a command's footprint from its precondition; in return the modality frames facts that overlap what the command writes. A precise correspondence would embed assertions about a subheap into assertions about the whole heap and relate $P * R$ to $\Box_\Phi R$ for the effect Φ confined to P's subheap; we state none. Whether the model is an instance of abstract separation logic [6] or of Views [7] is open; it would need a composition on supports.

The closest relatives are region logic and dynamic frames, for effects as region expressions read in the pre-state; rely/guarantee with explicit stabilisation, for the modality; and monotone state [2, 3], for init and grow, which here are enforced by the effect judgment and by no resource discipline.

How much separation. In these patterns: pairwise disjointness of single stores (#), and of a store with its cells (apart) where a fact reads element contents, each established by allocation or assumed at a function's entry. Five of the eleven obligations use none. This describes the patterns met so far and is no completeness claim.

10 Limits

Effects are closed

An effect has no variables. So a higher-order function states one bound for every callback it admits [5].

Await

Across an `await` the effect is that of the environment, `⊤`.

Broken and restored invariants

An invariant that a callee breaks and then restores has no form. At each step a fact either enters the modality or is derived anew.

Fixed contracts

A location's contract never changes, so there is no type-changing update.

Transfer as one Lean theorem

The laws and rules are stated for admissible transitions and use the properties (T1) to (T4) of Section 3. Theorem 13 is not one Lean theorem. Stating it in Lean needs, beyond those properties, the correspondence between this note's named variables and the model's environment, which a result extends by a push.

References

[1] A. Ahmed. *Semantics of Types for Mutable State*. PhD thesis, Princeton University, 2004.

[2] A. Ahmed, D. Dreyer, A. Rossberg. State-dependent representation independence. POPL 2009.

[3] D. Ahman, C. Fournet, C. Hriţcu, K. Maillard, A. Rastogi, N. Swamy. Recalling a witness: foundations and applications of monotonic state. PACMPL 2(POPL), 2018.

[4] A. Banerjee, D. Naumann, S. Rosenberg. Local reasoning for global invariants, part I: region logic. J. ACM 60(3), 2013.

[5] L. Birkedal, N. Torp-Smith, H. Yang. Semantics of separation-logic typing and higher-order frame rules. LICS 2005.

[6] C. Calcagno, P. O'Hearn, H. Yang. Local action and abstract separation logic. LICS 2007.

[7] T. Dinsdale-Young, L. Birkedal, P. Gardner, M. Parkinson, H. Yang. Views: compositional reasoning for concurrent programs. POPL 2013.

[8] C. B. Jones. Tentative steps toward a development method for interfering programs. ACM TOPLAS 5(4), 1983.

[9] I. Kassios. Dynamic frames: support for framing, dependencies and sharing without restrictions. FM 2006.

[10] P. O'Hearn, J. Reynolds, H. Yang. Local reasoning about programs that alter data structures. CSL 2001.

[11] J. Wickerson, M. Dodds, M. Parkinson. Explicit stabilisation for modular rely-guarantee reasoning. ESOP 2010.